

EduGenie: Google Gemini Powered Learning Assistant

Team ID: SWTID-2026-8809

Project Description:

EduGenie is a lightweight AI-powered educational assistant that simplifies learning through generative AI. Designed for students of all academic levels, EduGenie enables users to ask questions and receive smart, concise answers, understand complex concepts through simplified explanations, generate quizzes from topics or text, receive personalized learning recommendations, and summarize large educational passages.

Built with FastAPI for the backend and a simple HTML + CSS frontend, EduGenie leverages lightweight and cloud-based AI models for local efficiency and cloud power. Gemini 1.5 Pro (via API) powers Q&A, summarization, quiz generation and learning paths, while the LaMini-Flan-T5-783M model runs locally for concept explanation. It works well on devices like the Mac M1, making it accessible to a broad range of learners and developers.

Scenarios

Scenario 1: A student who wants to know about oceans and rivers uses EduGenie to ask “Which is the largest ocean?” The Q&A module sends the question to Gemini 1.5 Pro and shows a short, accurate answer right below the input box.

Scenario 2: A student who wants to check her understanding of “The Pythagoras Theorem” clicks “Generate Quiz.” EduGenie creates three multiple-choice questions with four options each and corrects the answer if a wrong option is chosen.

Scenario 3: A learner exploring SQL requests a learning path. EduGenie returns a structured plan with beginner to advanced topics, timelines and suggested resources such as videos, articles and books.

Technical Architecture:

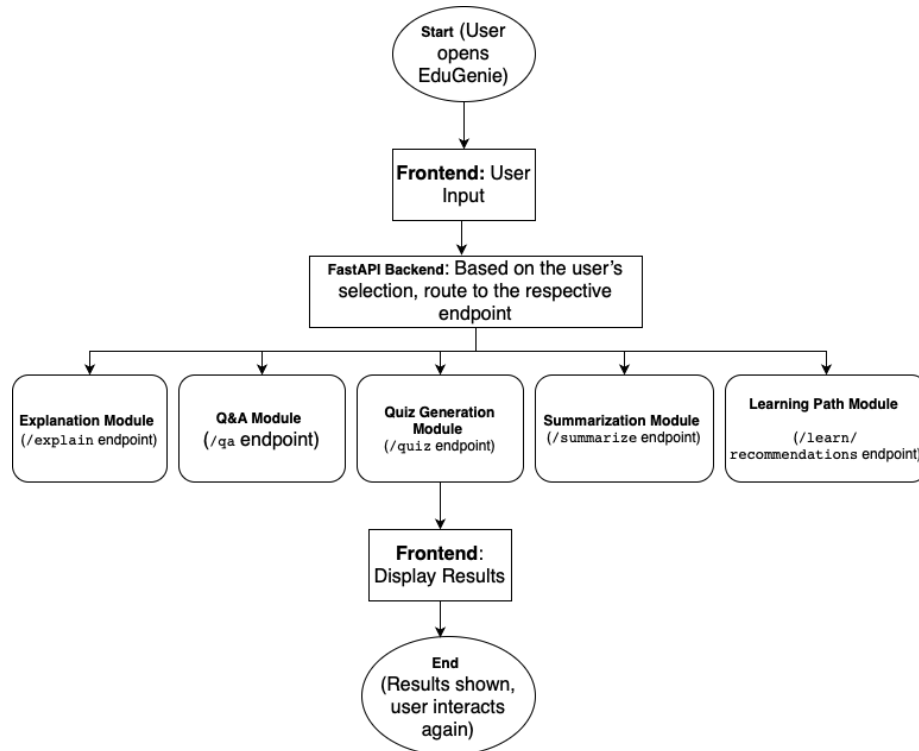


Fig. EduGenie technical architecture and request flow

Description: EduGenie follows a modular architecture. The HTML + CSS frontend collects the user's input and sends it to the FastAPI backend, which routes the request to the right module: Explanation (LaMini-Flan-T5, local), Q&A, Quiz, Summarization and Learning Path (Gemini 1.5 Pro, via API). The result is sent back to the frontend and displayed below the input box. The application runs locally with Uvicorn, and the Gemini API key is managed securely.

(Note: EduGenie does not use a database, as no user data is stored, so an ER Diagram is not required.)

Pre-requisites:

- **Python Programming Proficiency (Python 3.10+):** [Python Documentation](#)
- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **HTML and CSS Skills:** Basic templating used in /templates and /static
- **Google Gemini API Key:** [Google AI for Developers](#)
- **Uvicorn (ASGI Server):** [Uvicorn Documentation](#)
- **Jinja2 (HTML Templating Engine):** [Jinja2 Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Select the AI models for the project (Gemini 1.5 Pro and LaMini-Flan-T5).
- **Activity 1.2:** Define the folder architecture of the application.
- **Activity 1.3:** Generate a Gemini API key and store it securely.
- **Activity 1.4:** Set up the development environment and install the required libraries.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Implement the Explanation, Q&A, Quiz, Summary and Learning Path modules.
- **Activity 2.2:** Build the backend API with FastAPI and connect each endpoint to its module.

Milestone 3: Frontend Development

- **Activity 3.1:** Build the web interface using HTML and CSS.
- **Activity 3.2:** Integrate the frontend with the backend for live results.

Milestone 4: Deployment

- **Activity 4.1:** Run the application locally using Uvicorn.
- **Activity 4.2:** Perform functional testing of all modules.

Milestone 5: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we select the AI models that power EduGenie and define how the application is organized. A combination of a cloud-based model and a lightweight local model keeps the application both powerful and easy to run on ordinary devices.

Activity 1.1: Select AI Models

- **Gemini 1.5 Pro (via API):** Used for Q&A, summarization, quiz generation and learning paths. Benefits: advanced reasoning, structured outputs and cloud inference.
- **LaMini-Flan-T5-783M (local):** Used for concept explanation. Benefits: instruction-tuned, lightweight and CPU-compatible.

Activity 1.2: Define the Folder Architecture

The project is organized into separate files for each module, along with the templates and static files for the frontend.

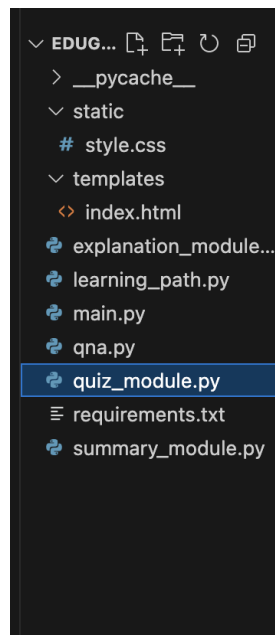


Fig. Folder architecture of EduGenie

- **main.py:** FastAPI application and API endpoints.
- **explanation_module.py, qna.py, quiz_module.py, summary_module.py, learning_path.py:** logic for each module.
- **templates/index.html and static/style.css:** HTML frontend and styling.
- **requirements.txt:** Python dependencies.

Activity 1.3: Generate a Gemini API Key

Before the application can use Gemini 1.5 Pro, you need a valid Gemini API key. Follow the steps below.

- **Step 1:** Visit Google AI Studio and sign in with your Google account.
- **Step 2:** Create a new project and enable the Gemini API.

- **Step 3:** Generate an API key, copy it and store it securely. Do not share the key or upload it to a public repository.

Activity 1.4: Set Up the Development Environment

- **Install Python 3.10+:** Download the latest installer from the official website, run it and check “Add Python to PATH”, then follow the prompts.
- **Verify the installation:** Open a terminal or command prompt and type:

```
> python --version
```

- **Install the libraries:** Install FastAPI, Uvicorn and Jinja2 along with the AI libraries:

```
> pip install fastapi uvicorn jinja2  
> pip install google-generativeai transformers torch
```

Milestone 2: Core Functionalities Development

Milestone 2 focuses on building the five learning modules and exposing them through a FastAPI backend.

Activity 2.1: Module Implementation

Explanation Module

EduGenie uses the LaMini-Flan-T5 model, a lightweight yet powerful generative model, to deliver educational content in a simplified and highly readable manner. It is fine-tuned to give concise, context-aware responses that break complex topics into easy language, so learners with minimal background can grasp essential concepts without feeling overwhelmed.

```

explanation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
  
```

Fig. Explanation module logic

Q&A Module

EduGenie is powered by Gemini 1.5 Pro, enabling it to handle a wide range of general knowledge and academic question-answering tasks with precision.

```

def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in QnA: {e}"
  
```

Fig. Q&A module

Quiz Module

EduGenie generates three multiple-choice questions (MCQs) from a given passage, each with four options. Gemini understands the context so that the questions are relevant and the wrong options are believable. The output is in JSON format for easy integration. The prompt expects a valid JSON response, markdown code blocks are cleaned using the clean_json_block function, and the cleaned response is parsed into a Python list. If an error occurs, a detailed error message is returned.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(.*?)```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> List:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"""
        You are a quiz generator.

        From the following passage, create 3 multiple-choice questions. Each question should include:
        - A "question"
        - A list of 4 "options"
        - A correct "answer" that must exactly match one of the options.

        Format your output as **valid JSON**, like this:
        [
          {{
            "question": "What is ...?",
            "options": ["A", "B", "C", "D"],
            "answer": "A"
          }}
        ]

        Passage:
        {text}
        """
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧹 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)
    except Exception as e:
        return f"⚠️ Error in Summary: {e}"
```

Fig. Quiz module logic

Summary Module

This feature uses Gemini to summarize long paragraphs into short, easy-to-understand versions, ideal for quick revision. It keeps the core information while removing repetition.

```
def summarize_text(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in Summary: {e}"
```

Fig. Summary module

Learning Path Module

The `get_learning_recommendations` function uses Gemini to generate a personalized, structured learning path for any topic. The prompt asks for beginner to advanced concepts organized by difficulty, supported by resources such as videos, articles or books. API responses and errors are handled safely, returning either the generated text or a helpful error message.

```
def get_learning_recommendations(topic):
    prompt = f"""
    You are an AI tutor. The student wants to learn about: {topic}.
    Suggest a structured and adaptive learning path including key topics, order of learning, and resources (1
    Include beginner, intermediate, and advanced levels if needed.
    """
    try:
        response = model.generate_content(prompt)
        print("🗨️ Gemini raw response:", response)

        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "❌ Could not extract content from Gemini response."
    except Exception as e:
        import traceback
        traceback.print_exc() # ✅ This gives full error trace
        return f"❌ Error occurred: {str(e)}"
```

Fig. Learning path module

Activity 2.2: Backend API with FastAPI

RESTful endpoints were defined for each module and connected to the respective module logic:

- **/qa** for question answering
- **/explain** for concept explanation
- **/quiz** for quiz generation
- **/summarize** for summarization
- **/learn/recommendations** for learning paths

```
main.py
40
41 # Q&A - GET API using Gemini
42 @app.get("/qa")
43 async def answer_question(question: str = Query(...)):
44     answer = answer_question_with_gemini(question)
45     return {"answer": answer}
46
47 # Explanation - POST API
48 @app.post("/explain/")
49 async def explain_api(request: Request):
50     data = await request.json()
51     topic = data.get("topic")
52     if not topic:
53         return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
54     explanation = explain_topic(topic)
55     return {"topic": topic, "explanation": explanation}
56
57 # Summarization - POST API
58 @app.post("/summarize/")
59 async def summarize_api(request: Request):
60     data = await request.json()
61     text = data.get("text")
62     if not text:
63         return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
64     summary = summarize_text(text)
65     return {"summary": summary}
66
```

Fig. Endpoints of API (Q&A, Explanation and Summarization)

```
66
67 # Quiz Generation - POST API
68 @app.post("/quiz")
69 async def quiz_api(request: Request):
70     data = await request.json()
71     text = data.get("text")
72     if not text:
73         return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
74     quiz = generate_quiz(text)
75     print("Generated quiz:", quiz) # [X] DEBUG
76     return JSONResponse(content={"quiz": quiz})
77
78 # Learning Recommendations - GET API
79 @app.get("/learn/recommendations")
80 async def learning_recommendation_api(topic: str = Query(...)):
81     recommendation = get_learning_recommendations(topic)
82     return {"topic": topic, "recommendation": recommendation}
83
84
```

Fig. Endpoints of API (Quiz and Learning Recommendations)

Milestone 3: Frontend Development

Milestone 3 focuses on building a simple, clean and responsive web interface for EduGenie and connecting it to the backend.

Activity 3.1: Build the Web Interface

- **HTML Form (index.html):** Input areas and buttons for each task (Ask, Explain, Summarize, Quiz and Recommend), a text area for user input and a submit button.
- **CSS Styling (style.css):** Responsive design, styled buttons and input areas, and a clear result container.

```

templates > <> index.html > ...
 2  <html lang="en">
 3  <head>
86  </head>
87  <body>
88    <h1>Welcome to EduGenie 🤖</h1>
89    <p>Your personal AI tutor for learning support!</p>
90
91    <!-- Q&A -->
92    <form id="qaForm">
93      <label><strong>Ask EduGenie a Question:</strong></label><br><br>
94      <input type="text" id="question" placeholder="Why is the sky blue?" required>
95      <button type="submit">Get Answer</button>
96    </form>
97    <div id="qaResult" class="output"></div>
98
99    <!-- Explanation -->
100   <form id="explainForm">
101     <label><strong>Need an Explanation?</strong></label><br><br>
102     <input type="text" id="topic" placeholder="Photosynthesis" required>
103     <button type="submit">Explain</button>
104   </form>
105   <div id="explanationResult" class="output"></div>
106
107   <!-- Summary -->
108   <form id="summaryForm">
109     <label><strong>Summarize a Paragraph:</strong></label><br><br>
110     <input type="text" id="summaryText" placeholder="Paste long content to summarize" required>
111     <button type="submit">Summarize</button>
112   </form>
113   <div id="summaryResult" class="output"></div>
114
115   <!-- Quiz -->
116   <form id="quizForm">
117     <label><strong>Generate a Quiz:</strong></label><br><br>
118     <input type="text" id="quizText" placeholder="Solar System" required>
119     <button type="submit">Generate Quiz</button>
120   </form>
121   <div id="quizResult" class="output"></div>

```

Fig. Frontend code (index.html)

Activity 3.2: Live Integration

- The form submission sends a POST request to the FastAPI backend.
- The result appears below the input box in real time.

Milestone 4: Deployment

In Milestone 4, the application is run locally to verify that every module works correctly.

Activity 4.1: Run Locally

- Open a terminal in the project folder and start the server using Uvicorn:

```
> uvicorn main:app --reload
```

- Open a browser and navigate to **http://127.0.0.1:8000** to access the application.

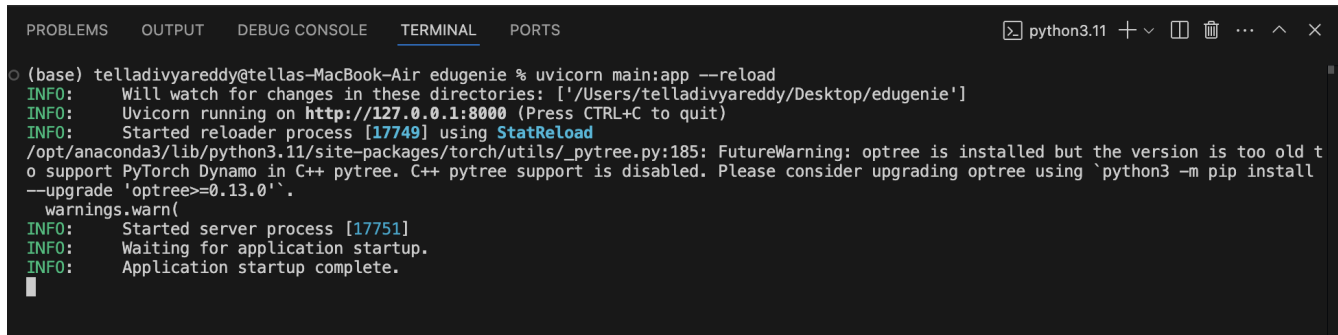


Fig. The application running locally

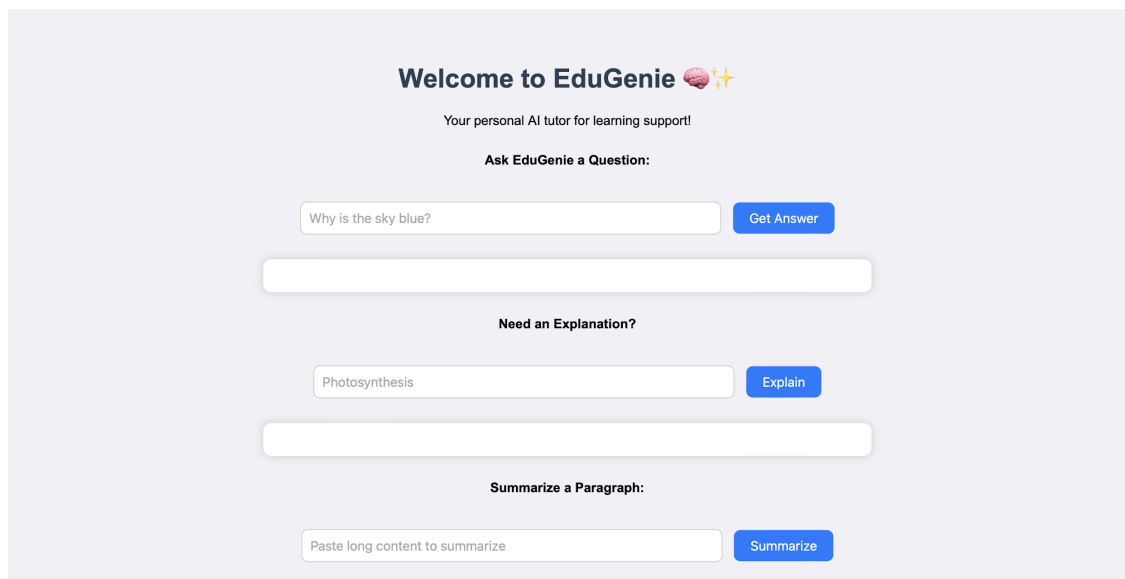
Activity 4.2: Functional Testing

All features of EduGenie were tested to make sure they work as expected:

- Ask a question and check the answer.
- Get an explanation of a concept.
- Generate a quiz and check the answer feedback.
- Summarize a long passage.
- Get a personalized learning plan.

Exploring the Website's Web Pages:

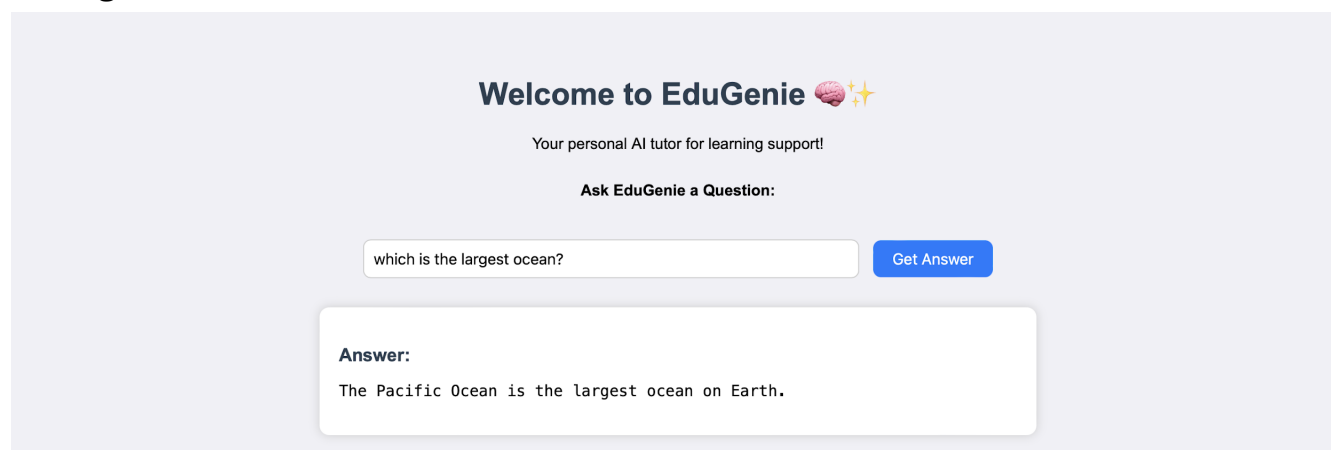
Home / Generator Page:



The screenshot shows the EduGenie interface. At the top, it says "Welcome to EduGenie" with a brain and star icon, followed by the tagline "Your personal AI tutor for learning support!". Below this is a section titled "Ask EduGenie a Question:" with a text input field containing "Why is the sky blue?" and a blue "Get Answer" button. Underneath is a long white box for the answer. The next section is "Need an Explanation?" with a text input field containing "Photosynthesis" and a blue "Explain" button, followed by another long white box. The final section is "Summarize a Paragraph:" with a text input field containing "Paste long content to summarize" and a blue "Summarize" button, followed by a long white box for the summary.

Description: The single-page interface of EduGenie is both the landing page and the learning tool. It has a welcome header with the tagline “Your personal AI tutor for learning support,” and sections for asking a question, getting an explanation, summarizing a paragraph, generating a quiz and getting learning recommendations.

Asking Questions:



This screenshot shows the "Ask EduGenie a Question:" section. The text input field contains the question "which is the largest ocean?" and the blue "Get Answer" button is clicked. Below the input field, the answer is displayed in a white box: "Answer: The Pacific Ocean is the largest ocean on Earth."

Description: The learner types a question, for example “Which is the largest ocean?”, and clicks the button. The answer from Gemini 1.5 Pro appears in an answer box below the input.

Explanation of Any Topic:

Need an Explanation?

Explanation:

Quantum computing is a type of computing that uses the principles of quantum mechanics to perform certain calculations much faster than traditional computers. It allows computers to perform certain calculations much faster than traditional computers because it uses tiny particles called qubits that can exist in multiple states at the same time. This allows for new kinds of calculations and solving problems that are not possible with traditional computers.

Description: The learner enters a topic such as “quantum computing.” The local LaMini-Flan-T5 model returns a short, simple explanation.

Summarizing Long Paragraphs:

Summarize a Paragraph:

Summary:

The Industrial Revolution changed the world by shifting it from farming to factories. New machines, like James Watt's steam engine, made things faster and easier to produce. While this boosted economies, it also created problems like bad working conditions, child labor, pollution, and overcrowded cities. Even with these problems, the Industrial Revolution shaped the modern world, affecting how we travel, communicate, work, and trade.

Description: A long passage is pasted into the input box and EduGenie returns a short summary that keeps the key points, which is useful for quick revision.

Generating Quizzes:

Generate a Quiz:

Quiz:

Q1: What does the Pythagorean theorem describe?

- ☐ The relationship between the angles of a triangle.
- ☐ The relationship between the sides of a right-angled triangle.
- ☐ The relationship between the area and perimeter of a triangle.
- ☐ The relationship between the sides of any triangle.

Q2: If 'a' and 'b' are the lengths of the two shorter sides of a right-angled triangle, and 'c' is the length of the longest side (hypotenuse), what equation represents the Pythagorean theorem?

- ☐ $a + b = c$
- ☐ $a^2 + b^2 = c^2$
- ☒ $a^2 - b^2 = c^2$
- ☐ $2a + 2b = 2c$

✗ Incorrect. Correct answer: $a^2 + b^2 = c^2$

Q3: Which type of triangle does the Pythagorean theorem apply to?

- ☐ Equilateral triangles
- ☐ Isosceles triangles
- ☒ Right-angled triangles
- ☐ All types of triangles

✔ Correct!

Description: EduGenie generates three questions with four options each. When a wrong option is chosen, it shows the correct answer.

Learning Recommendations:

Get Learning Recommendations:

Get Recommendations

Learning Recommendations for "SQL":

SQL Learning Path: From Zero to Hero

This learning path is structured to progressively introduce SQL concepts, starting from the basics and gradually advancing to more complex topics. It's designed to be adaptive: feel free to adjust the pace and delve deeper into areas that particularly interest you.

****I. Beginner Level: Building a Foundation****

****(Estimated Time: 1-2 weeks)****

*** **Key Topics:****

- * What is a Database and SQL? (Relational Model, DBMS)
- * Basic Syntax (SELECT, FROM, WHERE)
- * Data Types (INT, VARCHAR, DATE, etc.)
- * Filtering Data (WHERE clause with comparison operators, logical operators: AND, OR, NOT)
- * Ordering Results (ORDER BY)
- * Limiting Results (LIMIT/OFFSET or TOP)
- * Basic Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

Description: The learner enters a topic such as SQL and receives a path from beginner to advanced level, with details of resources to learn from and step-by-step guidance throughout.

Conclusion:

EduGenie is an AI-powered educational assistant that combines cloud-based intelligence with a simple, low-footprint web interface. It simplifies complex concepts, answers questions instantly, generates quizzes, summarizes passages and builds personalized learning paths, so that even users with minimal hardware can access high-quality learning support. The project covered model selection, module development, frontend design and local deployment, and gave valuable experience in prompt engineering, cloud integration and user-centric design. Looking ahead, EduGenie can be extended with voice-based interaction, multilingual support, a mobile app, progress tracking dashboards, gamified elements such as badges and streaks, adaptive learning paths, LMS integration (such as Moodle or Google Classroom) and image or PDF input. With these upgrades, EduGenie can grow into a complete learning companion for learners of all levels.

Team Work Allocation

S.No	Team Member	Work Assigned
1	Divya K	Project Description, Scenarios, Technical Architecture and Pre-requisites
2	Veronica	Milestone 1 (Model Selection and Architecture) and Milestone 2 (Core Functionalities Development)
3	Sangeetha T	Milestone 3 (Frontend Development)
4	Pooja N	Milestone 4 (Deployment and Functional Testing)
5	Monika N	Exploring the Website's Web Pages and Conclusion